

**UNICC AI Safety Lab Capstone Part (B): Fine-Tuning the SLM and Building
the
Council of Experts**

Applied Technical Project
Final Report

by

Dongdi Sun
DS8154

Spring 2026

A paper submitted in partial fulfillment of the requirements for the degree of

Master of Science

at the

Division of Programs in Business

School of Professional Studies

New York University

Abstract

This applied project delivered Part (B) of the UNICC AI Safety Lab Capstone by designing and implementing a multi-module inference ensemble — termed the Council of Experts — built on a domain-adapted Small Language Model (SLM) deployed on the NYU DGX Spark cluster. The work addressed a critical gap in AI governance: existing evaluation frameworks provide taxonomies of risk but do not operationalize pre-deployment, multi-perspective safety evaluation in an auditable, on-premises environment. Three independent expert modules were implemented — Judge 1 (Technical Safety), Judge 2 (Governance and Policy Alignment), and Judge 3 (Operational Risk) — each fine-tuned using Low-Rank Adaptation (LoRA) on synthetically generated governance, risk, and compliance (GRC) datasets. An orchestration layer coordinates inter-judge critique and disagreement analysis, and a synthesis engine produces a final risk verdict with recommended actions. The system was validated against VeriMedia, a real-world Flask-based disinformation detection service, producing divergent single-judge assessments (Medium, Critical, and High) that were reconciled into a unified Critical verdict with a council decision of REJECT. Benchmark evaluation revealed that while the base llama3.2:3b model achieved strong baseline performance (risk-tier match rates of 90% and 100% for Judges 1 and 2 respectively), the LoRA fine-tuned models produced schema-invalid outputs due to a mismatch between training data format and the Pydantic evaluation schema — a finding that constitutes a substantive lesson for future SLM deployment work under constrained compute budgets.

Table of Contents

1. Introduction	6
2. Problem Description / Opportunity	7
3. Importance of the Project to the Organization	9
4. Alternate Solutions Evaluated	10
5. Solution Evaluation Criteria	11
6. Selection Rationale	12
7. Approach and Methodology	12
8. Project Objectives and Metrics for Project Evaluation	14
9. Risk Analysis	15
10. Issues Encountered	16
11. Detailed Project Chronology and Critique	17
12. Lessons Learned	22
13. Conclusions and Summary	24
14. Limitations, Recommendations and Scope for Future Work	25
15. Literature Survey	28
16. Situational Analysis	37
Appendices	
Appendix A. Project Status Reports (Midterm Yellow — Mar 17, 2026; Final Green — Apr 17, 2026)	45
Appendix B. Project Plan, Baseline WBS	48
Appendix C. References and Bibliography (APA)	49

Acknowledgements

I sincerely thank Dr. Andres Fortino for serving as my Applied Technical Project Sponsor and the United Nations International Computing Centre (UNICC) for providing the opportunity to conduct this Applied Technical Project. I also thank Prof. Israel Moskowitz for the Applied Technical Project course and mentoring. I am grateful to my teammates Jingyi Weng and Qing Wang for their close collaboration throughout the project, and I appreciate the feedback from classmates and faculty that supported milestone improvements.

Declaration

I grant powers of discretion to the Department, SPS, and NYU to allow this paper to be copied in part or in whole without further reference to me. This permission covers only copies made for study purposes or for inclusion in Department, SPS, and NYU research publications, subject to normal conditions of acknowledgement.

1. Introduction

This Applied Technical Project was conducted for the United Nations International Computing Centre (UNICC) as part of the NYU SPS Applied Project course (MASY GC-4100). UNICC has over 50 years of experience as the largest strategic partner for digital solutions and cybersecurity within the United Nations system. In this environment, AI systems are evaluated not only for technical capability but also for trustworthiness, auditability, governance alignment, and the ability to produce consistent, explainable results across multiple stakeholder perspectives. A prototype that performs well in isolation but cannot be audited, reproduced, or understood by non-developer stakeholders is not considered deployable.

This project focused on Part (B) of the UNICC AI Safety Lab Capstone: fine-tuning a Small Language Model (SLM) and building a multi-module inference ensemble — the Council of Experts — to evaluate AI agents within the UNICC AI Sandbox before production deployment. The project sponsor was Dr. Andres Fortino, who provided expectations and acceptance criteria for the Part (B) deliverables. The project timeline ran from February 1, 2026 through April 14, 2026 (practical completion), with the final status report submitted on April 17, 2026.

Part (B) sits at the core of the three-part capstone: Part (A) established the SLM platform infrastructure on the NYU DGX Spark cluster, and Part (C) delivered the user-facing interface and deployment reproducibility package. Part (B) bridges these two by implementing the evaluation intelligence: the three expert judges, the critique arbitration layer, the synthesis engine, and the LoRA fine-tuning pipeline. Without Part (B), the platform has no evaluation logic and the interface has no results to display. The success of the integrated system therefore depends critically on the quality, reliability, and schema-compliance of the outputs produced by the Council of Experts.

The project prioritized three outcomes: (1) a functioning end-to-end evaluation pipeline in which three independent judges produce structured, Pydantic-validated assessments; (2) a domain-adapted SLM through LoRA fine-tuning on GRC-specific datasets; and (3) a critique and synthesis layer that surfaces inter-judge disagreements and produces a reconciled final verdict with actionable recommendations. All three outcomes were achieved, though the fine-tuning evaluation revealed a significant schema-alignment issue that is documented as a key lesson for future work.

2. Problem Description / Opportunity

The deployment of AI agents in institutional settings — particularly within the United Nations system — introduces a class of safety and governance challenge that existing tools do not adequately address. Most AI safety evaluation approaches fall into one of three categories: (1) framework-level guidance such as NIST AI RMF, OWASP LLM Top 10, and MITRE ATLAS, which provide taxonomies of risks but do not operationalize evaluation in a deployable tool; (2) commercial observability platforms such as Fiddler, which focus on post-deployment runtime monitoring rather than pre-deployment gatekeeping; or (3) general-purpose red-teaming tools that test specific vulnerability classes in isolation rather than integrating technical, governance, and operational risk dimensions into a unified verdict.

None of these approaches simultaneously provides pre-deployment scope, multi-perspective

integration, domain adaptation for institutional contexts, and on-premises auditable infrastructure. This gap is particularly consequential for UNICC, which operates across jurisdictions with differing data protection regimes and cannot rely on external APIs or black-box systems for safety evaluation. The organization requires a system that can evaluate AI agents before deployment, produce transparent and auditable results, and do so without transmitting sensitive data to external services.

A further challenge identified in empirical research is that single-perspective evaluation is epistemically insufficient for AI safety. A system that scores well on technical robustness may fail on governance alignment; a system with good adversarial resilience may still violate GDPR data minimization requirements. Cisco's 2026 State of AI Security report found that multi-turn jailbreak success rates on popular open-weight models reached as high as 92.78%, demonstrating that model-level alignment alone is inadequate and that evaluation must be both multi-dimensional and stress-tested across extended interaction sequences.

The opportunity for Part (B) was to fill this gap by building a Council of Experts ensemble that evaluates every AI agent submission from three independent perspectives — technical safety, governance and policy alignment, and operational risk — and synthesizes their assessments through explicit critique and arbitration into a final, actionable verdict. Deploying this on the NYU DGX Spark cluster using open-weight models eliminates external API dependencies and satisfies UNICC's data sovereignty requirements.

3. Importance of the Project to the Organization

For UNICC and UN-related environments, the ability to evaluate AI systems before deployment is a prerequisite for responsible adoption. The organization's mandate encompasses digital services for UN entities operating under diverse regulatory frameworks — including GDPR, the EU AI Act, and various national data protection laws — making multi-jurisdictional compliance a non-negotiable baseline for any deployed AI agent. A pre-deployment evaluation system that can assess agents across technical, governance, and operational dimensions provides UNICC with the assurance it needs to adopt AI capabilities without incurring unacceptable governance risk.

Part (B) directly advances this organizational need. The Council of Experts architecture makes evaluation multi-perspective by design: because the three judges are independent and each covers a distinct risk dimension, no single failure mode can be obscured by strong performance in another area. The critique round surfaces disagreements between judges explicitly, encouraging human reviewers to examine edge cases rather than accepting an aggregate score uncritically. The synthesis engine produces actionable next steps — not merely a score — so that development teams know what remediation is required before resubmission.

The on-premises deployment model also has strategic value for UNICC beyond this specific capstone. By demonstrating that a meaningful AI safety evaluation capability can be built on open-weight SLMs running on local infrastructure, this project establishes a replicable pattern for governance-aligned AI adoption that does not depend on external vendors. The LoRA fine-tuning pipeline, the GRC dataset generation tooling, and the modular judge architecture are all assets that future teams can extend as evaluation requirements evolve.

4. Alternate Solutions Evaluated

Several alternatives were considered for implementing the Council of Experts under the constraints of the NYU DGX Spark environment and the capstone timeline.

The first alternative was a single-judge evaluation system using a single LLM prompted to cover all risk dimensions simultaneously. This approach reduces implementation complexity and eliminates the need for an orchestration layer. However, it sacrifices the epistemic diversity that is the central motivation for the council architecture: a single model applying a single prompt cannot reliably hold technical, governance, and operational risk dimensions simultaneously without inter-dimensional trade-offs. It also provides no mechanism for surfacing disagreements, which is one of the most valuable outputs of the ensemble approach.

The second alternative was to use external API-based LLMs (such as GPT-4o or Claude) as the evaluation backend, with locally implemented judge logic. This would have provided higher raw capability for judgment tasks, but it violates UNICC's data sovereignty requirement that sensitive AI agent submissions not be processed by external vendors. It also introduces dependency on API availability, rate limits, and cost — none of which are acceptable in a governance-critical evaluation pipeline.

The third alternative was to use a rule-based evaluation system with no LLM involvement — purely regex pattern matching and keyword detection. This approach is fully deterministic and auditable, but it cannot capture the contextual reasoning required for governance assessment. A rule-based system cannot, for example, determine whether a particular deployment context makes an unauthenticated upload endpoint a GDPR violation, because that determination requires understanding the combination of deployment context, data type, and regulatory framework.

The selected approach — three LoRA fine-tuned expert modules running on llama3.2:3b via Ollama, coordinated by an orchestrator and synthesized into a final verdict — offered the best balance of multi-perspective coverage, domain adaptation, on-premises deployment, and structured output reliability. Although the fine-tuning evaluation revealed schema-alignment issues (discussed in Section 10), the base model's strong performance confirmed that the architectural approach is sound and that the schema issue is an engineering problem with a clear remediation path.

5. Solution Evaluation Criteria

The solution was evaluated against five criteria:

- Multi-perspective coverage: the system must evaluate AI agents across technical safety, governance and policy alignment, and operational risk dimensions independently.
- Structured output reliability: each judge must produce Pydantic-validated JSON outputs that can be programmatically consumed by the orchestrator and synthesis layers.
- Domain adaptation: the SLM must demonstrate improved performance on GRC-specific evaluation tasks relative to the untuned baseline.

- On-premises deployability: the system must run without external API dependencies, using open-weight models on the DGX Spark cluster or equivalent local infrastructure.
- End-to-end integration: the evaluation pipeline must accept a structured submission input and produce a final council verdict with risk tier, reconciled score, and recommended next actions.

6. Selection Rationale

The Council of Experts architecture with LoRA fine-tuning was selected because it is the only approach that simultaneously satisfies all five evaluation criteria. The three-judge structure ensures multi-perspective coverage by design. The Pydantic validation layer enforces structured output reliability at runtime, with graceful error handling for schema failures. The LoRA fine-tuning pipeline supports domain adaptation without requiring full retraining, making it feasible under the DGX Spark cluster's compute budget. The Ollama deployment model eliminates external API dependencies. And the FastAPI backend with orchestrator and synthesis modules provides end-to-end pipeline integration that is both modular and extensible.

The selection was also reinforced by alignment with the broader capstone architecture: Part (A) delivers the SLM platform, Part (B) delivers the evaluation intelligence, and Part (C) delivers the user-facing interface. The Council of Experts approach produces structured JSON artifacts at every stage of the pipeline — judge outputs, critique round, synthesis output, and pipeline log — which are the exact format required by the Part (C) interface to render results to stakeholders.

7. Approach and Methodology

A milestone-driven approach was used to structure work into seven work packages: architecture and interface planning, expert layer development, microservice encapsulation, coordination layer development, SLM fine-tuning and integration, deployment and containerization, and project management and documentation. Cross-team dependency risk with Parts (A) and (C) was managed through an explicit integration contract — agreed-upon API schemas (Pydantic models: TestRequest, EvaluationResponse) and sample outputs — so that Part (B) development could proceed independently while remaining compatible with upstream and downstream interfaces.

Expert Module Design

Each of the three expert judges was implemented as an independent Python module that calls the local SLM via Ollama and returns a structured JSON assessment. Judge 1 (`judge1.py`, `AutoEvaluator`) covers nine protocols across two suites: Suite A core benchmarks (bias, robustness, transparency, explainability, privacy documentation) and Suite B adversarial protocols (evasion, poisoning, privacy inference, red-team). Each protocol produces a score, outcome, finding, and rationale. Judge 2 (`judge2.py`, `RegulatoryLens`) applies a governance and policy lens across five weighted dimensions: harmfulness (weight 0.35), bias and fairness (0.20), legal compliance (0.25), transparency (0.10), and deception (0.10). Judge 3 (`judge3.py`, `AdversarialProbe`) conducts adversarial red-team testing, generating five test cases targeting the submitted agent and evaluating pass/fail outcomes for prompt injection, PII leakage, supply chain risk, jailbreak resilience, and deployment posture.

Orchestration and Synthesis

The orchestrator module (`orchestrator.py`) coordinates the full multi-judge evaluation pipeline. After all three judges produce their assessments, the orchestrator conducts a critique round in which agreement points, disagreement points, and arbitration notes are identified. A reconciled score and risk tier are produced before the output is passed to the synthesis engine. The synthesis module (`synthesis.py`) produces the final council verdict — risk tier (Low, Medium, High, or Critical), reconciled score (0–100), council decision (APPROVE, `CONDITIONAL_APPROVE`, or REJECT), human review requirement, rationale, and recommended next actions. All pipeline artifacts are written to structured JSON files for audit purposes.

SLM Fine-Tuning

The fine-tuning pipeline targeted `llama3.2:3b` as the base model, selected for its open-weight availability, permissive licensing compatible with the AI Digital Public Good standard, and feasibility on the DGX Spark cluster's compute budget. Training datasets were generated using `generate_grc_dataset.py`, which produces instruction-response pairs aligned with the evaluation protocols of each judge. LoRA (Low-Rank Adaptation) was applied via `train_lora.py`, updating only a small fraction of model weights to reduce memory and compute requirements. Separate merged 16-bit models were produced for Judge 1 and Judge 2. Benchmark evaluation was conducted using `benchmark_models.py` against held-out evaluation datasets (`judge1_eval.jsonl` and `judge2_eval.jsonl`), with the base model (Ollama-served `llama3.2:3b`) as the baseline comparator.

Deployment and Integration

The system was packaged for deployment using a Dockerfile and Makefile, enabling one-command startup (`make run`) targeting the DGX Spark or sandbox environment. A mock mode (`MOCK_MODE=1`) was implemented for CI and testing without live LLM dependencies, returning pre-computed outputs that accurately represent the pipeline's behavior. The FastAPI backend exposes a `POST /submit` endpoint that accepts structured submissions and returns the full multi-judge report. The Streamlit frontend (`demo.py`) connects to the FastAPI backend via HTTP and renders all judge and synthesis outputs dynamically.

8. Project Objectives and Metrics for Project Evaluation

Project Goal: Deliver Part (B) by implementing three independent expert judge modules fine-tuned on a domain-adapted SLM, building the Council of Experts orchestration and synthesis layers, and producing a fully integrated end-to-end AI safety evaluation pipeline (Part B only; excludes platform infrastructure and user interface).

Objective 1: Define layer interaction protocols and API data schemas with Parts (A) and (C) by Week 3; verified by schema document and team agreement.

Objective 2: Implement and encapsulate the core Compliance Judge module as an independent FastAPI microservice by Week 5, producing structured JSON governance evaluations across UN/UNESCO frameworks.

Objective 3: Implement all three expert judge modules (Technical Safety, Governance and Policy, Operational Risk) and the orchestration critique layer by Week 8, with end-to-end

pipeline validation on at least one test case.

Objective 4: Complete SLM fine-tuning using LoRA on GRC-specific datasets by Week 10, with benchmark evaluation comparing fine-tuned and baseline model performance on held-out evaluation sets.

Objective 5: Finalize deployment package (Dockerfile, Makefile, mock mode, test suite) and validate end-to-end integration with Parts (A) and (C) by end of semester.

9. Risk Analysis

Risk management focused on four categories of risk specific to SLM-based evaluation systems in constrained compute environments.

The highest risk was cross-team dependency risk with Part (A). Part (B) depends on the base SLM environment delivered by Part (A); a delay in that delivery directly blocks the fine-tuning phase. This risk was managed by pivoting early work to expert module development and microservice encapsulation — work that could proceed independently of the base model — so that when the SLM became available, only the fine-tuning phase remained. This strategy was validated by the midterm status report, which noted that the delay caused by a personal emergency on the Part (A) team did not significantly impact the overall schedule.

The second risk was schema alignment risk between the training data format and the Pydantic evaluation schemas. Because the judge evaluation schemas (`Judge1StructuredAssessment`, `Judge2StructuredAssessment`) are strict — requiring exact field names, nested object types, and enumerated literal values — any training data that uses different field structures will produce schema-invalid outputs after fine-tuning. This risk was not fully mitigated and materialized in the benchmark evaluation, as discussed in Section 10.

The third risk was compute and memory risk on the DGX Spark cluster. LoRA fine-tuning of a 3B-parameter model requires significant GPU memory, and the cluster environment imposes resource quotas. This was addressed by using parameter-efficient LoRA rather than full fine-tuning and by selecting `llama3.2:3b` specifically for its feasibility on the available compute budget.

The fourth risk was integration compatibility risk with Part (C). The user interface depends on the pipeline producing consistently structured JSON artifacts; changes to field names or output structure late in the timeline could break the interface. This was managed through the integration contract established in Week 3, which defined required output fields and was used as the binding specification for both Part (B) and Part (C) development.

10. Issues Encountered

Three significant issues were encountered and managed during the project.

The first issue was the delayed delivery of the base SLM environment from the Part (A) team, caused by a personal emergency experienced by a team member. This delay pushed the start of the fine-tuning phase back by approximately two to three weeks. The mitigation strategy was to reorder work: expert module development, microservice encapsulation, and coordination layer

implementation were accelerated, and fine-tuning was deferred until the base model became available. This reordering was successful — by the time the SLM was delivered, all other pipeline components were complete and only the fine-tuning phase remained. The final status report confirmed all milestones were achieved on schedule.

The second and most technically significant issue was the schema-alignment failure in the fine-tuned models. Benchmark evaluation using `benchmark_models.py` revealed that the LoRA fine-tuned versions of both Judge 1 and Judge 2 produced a schema validity rate of 0.00 — meaning every generated output failed Pydantic validation. Analysis of the error messages identified two root causes: (1) the training data used a nested submission wrapper structure and legacy protocol naming conventions (e.g., 'Prop F1', 'Prop R1') rather than the flat schema and enumerated protocol IDs ('bias', 'robustness') required by the evaluation Pydantic models; and (2) the fine-tuned models for Judge 2 returned integer scores rather than the nested object structures expected by the schema. This is a substantive finding: the base model (untuned llama3.2:3b served via Ollama) achieved risk-tier match rates of 90% and 100% for Judges 1 and 2 respectively, demonstrating that the evaluation approach is sound. The fine-tuning failure is an engineering issue — a training data format mismatch — with a clear remediation path (regenerating training data using the exact schema field names and types).

The third issue involved minor integration alignment challenges with Parts (A) and (C). Because the three parts were developed in parallel, small changes to field names and output structures occasionally required updates to the binding layer. These were managed through coordination checkpoints and were resolved without significant schedule impact.

11. Detailed Project Chronology and Critique

The project followed a seven-phase execution sequence aligned with the WBS work packages:

- Architecture and interface planning (Weeks 1–3): API schemas, risk scoring metrics, and cross-project interaction protocols were defined. The Pydantic models — `TestRequest` and `EvaluationResponse` — were established as the binding integration contract.
- Expert layer development (Weeks 3–5): The core Compliance Judge module was implemented and encapsulated as a FastAPI microservice. Governance frameworks — UN/UNESCO AI Ethics, EU AI Act, NIST AI RMF, and GDPR — were integrated into the weighted scoring algorithm.
- Orchestration and synthesis (Weeks 5–7): The orchestrator critique layer and synthesis engine were implemented, enabling multi-judge output reconciliation and final verdict generation.
- Part (A) delay mitigation (Weeks 6–7): Following the midterm status report, work was reordered to accelerate pipeline integration while awaiting SLM delivery.
- SLM fine-tuning (Weeks 7–9): Upon receiving the base SLM, the GRC dataset generation pipeline was executed and LoRA fine-tuning was applied to Judge 1 and Judge 2. Benchmark evaluation was conducted using the held-out evaluation datasets.

- Integration and validation (Weeks 9–10): End-to-end integration with Parts (A) and (C) was completed. The VeriMedia case study was used as the primary validation test case. The Streamlit frontend was connected to the FastAPI backend.
- Deployment and documentation (Weeks 10–12): Dockerfile and Makefile were finalized, mock mode was implemented, and the test suite was completed. All pipeline artifacts — judge outputs, critique round, synthesis output, full result, and pipeline log — were validated.

Expanded week-by-week narrative:

In Weeks 1–2, the project scope was confirmed and the work breakdown structure was established. The key scoping decision was to treat the integration contract as a first-class deliverable rather than an informal agreement. By defining Pydantic models at the outset, Part (B) established a specification that both constrained its own output format and gave Parts (A) and (C) a stable interface to build against.

In Weeks 2–3, the DGX Spark cluster environment was explored and the feasibility of LoRA fine-tuning on llama3.2:3b was confirmed. The decision to use Ollama as the model serving layer was made based on its local deployment model, open-source licensing, and compatibility with the DGX environment. The three-judge architecture was finalized, with each judge assigned a distinct evaluation perspective and protocol suite.

In Weeks 3–5, the Compliance Judge module — the predecessor to Judge 2 — was implemented and validated as a standalone FastAPI microservice. This was the midterm milestone: a working microservice that accepted a standardized text input, ran LLM-based evaluations across international AI governance frameworks, and returned structured JSON outputs. The midterm status report (March 17, 2026) confirmed this milestone was achieved, with the only issue being the pending SLM delivery from Part (A).

In Weeks 5–7, the orchestration and synthesis layers were implemented. The critique round mechanism was designed to surface both agreement and disagreement between judges explicitly, rather than silently averaging scores. This design choice reflects the insight — supported by ensemble theory and Cisco's empirical research on AI safety — that inter-judge disagreements contain information that a human governance reviewer needs to see. The synthesis engine was implemented to produce not only a final risk tier and score but also structured recommended next actions aligned with the specific failure modes identified by each judge.

In Weeks 7–9, the LoRA fine-tuning pipeline was executed following SLM delivery. Training datasets were generated using `generate_grc_dataset.py`, producing instruction-response pairs for each judge's protocol suite. Fine-tuning was applied using `train_lora.py`, and merged 16-bit models were produced for both Judge 1 and Judge 2. Benchmark evaluation using `benchmark_models.py` was then conducted against held-out evaluation sets. The results — a schema validity rate of 0.00 for both fine-tuned models — identified the training data format mismatch issue described in Section 10. Rather than halting the project, this finding was documented as a significant lesson and the base model (untuned llama3.2:3b) was retained for end-to-end system validation, given its strong baseline performance.

In Weeks 9–10, end-to-end system validation was conducted using the VeriMedia case study. VeriMedia is a Flask-based disinformation detection service using GPT-4o and OpenAI Whisper, deployed with unauthenticated file upload endpoints and no rate limiting. The three-judge ensemble produced the following assessments:

Judge	Risk Tier	Score	Confidence	Perspective
Judge 1 — AutoEvaluator	Medium	55/100	0.60	Technical Safety
Judge 2 — RegulatoryLens	Critical	91/100	0.82	Governance & Policy
Judge 3 — AdversarialProbe	High	74/100	0.70	Operational Risk
Final Council Verdict	CRITICAL	80/100	—	REJECT — Human Review Required

The divergent single-judge assessments — Medium, Critical, and High — illustrate precisely the epistemic value of multi-perspective evaluation. The critique round identified the divergence and escalated the final verdict to Critical, triggering a council decision of REJECT with a requirement for human governance board sign-off before any deployment activity. A single-judge system would have produced either an incomplete or a misleading assessment depending on which perspective was chosen.

In Weeks 10–12, the deployment package was finalized, mock mode was validated, and the test suite was completed. The Streamlit frontend was connected to the FastAPI backend via HTTP, enabling dynamic result rendering of all judge and synthesis outputs. The final status report (April 17, 2026) confirmed all milestones were achieved with no outstanding blockers.

Critique: The most significant improvement for future iterations would be schema-locking the training data generation pipeline to the evaluation Pydantic models at the point of dataset creation, so that any schema evolution automatically propagates to the training data. A schema contract test — a simple automated check that validates sample training outputs against the evaluation schema before fine-tuning begins — would have caught the mismatch before compute was spent on training. Additionally, a larger and more diverse training dataset would improve the fine-tuned model's generalization; the 10-example held-out evaluation sets used in this project are appropriate for a capstone prototype but insufficient for production deployment.

12. Lessons Learned

1. Schema alignment must be enforced at every stage of the ML pipeline.

The fine-tuning benchmark failure was caused by a mismatch between the training data format and the Pydantic evaluation schema — a problem that could have been caught before any training compute was spent. The lesson is that evaluation schemas should be treated as contracts

that propagate through the entire pipeline: data generation, training, and evaluation must all operate on the same schema version. In future work, a lightweight schema contract test run as part of dataset generation would detect format mismatches immediately.

2. Strong baseline performance is a valuable finding, not a consolation prize.

The untuned llama3.2:3b achieved risk-tier match rates of 90% and 100% for Judges 1 and 2 respectively, with schema validity rates of 100% in both cases. This finding demonstrates that the evaluation approach — structured prompting of an SLM via Ollama — is effective even without fine-tuning. For future iterations, this means that the highest priority is not fine-tuning per se, but prompt engineering and schema-aligned training data that allow fine-tuning to improve upon an already-strong baseline.

3. Multi-perspective evaluation reveals information that single-judge systems cannot.

The VeriMedia case study produced divergent assessments across three judges — Medium, Critical, and High — that were reconciled into a unified Critical verdict. Neither any individual judge nor a simple average would have produced this result. The lesson is that epistemic diversity is not merely a design preference but a functional requirement for safety evaluation in institutional settings: different risk dimensions can produce dramatically different verdicts, and surfacing that divergence is what allows human reviewers to make informed governance decisions.

4. Cross-team integration contracts should be established as early as possible and enforced programmatically.

The integration delay caused by the Part (A) team emergency was manageable because Part (B) had already established a clear integration contract and could develop independently. However, the contract was informal rather than programmatically enforced. A schema version file committed to the shared repository — with automated checks that fail when any part produces output that violates the contract — would have reduced integration friction and prevented the kind of field-name drift that required manual reconciliation at several points.

5. Reordering work in response to dependency delays is a professional project management skill.

When the base SLM delivery was delayed, the response was to accelerate work on the orchestration and synthesis layers — components that could be developed and tested with mocked judge outputs. This decision preserved project momentum and ensured that the integration phase could proceed immediately once the SLM was available. The lesson is that milestone-driven planning should always identify which work packages can proceed in parallel and which are on the critical path, so that delay responses are pre-planned rather than improvised.

6. Audit artifacts are a first-class deliverable, not an afterthought.

The decision to produce structured JSON artifacts at every stage of the pipeline — judge outputs, critique round, synthesis output, full result, and pipeline log — was motivated by UNICC's auditability requirements. In practice, these artifacts were also essential for debugging, for the VeriMedia case study write-up, and for the Part (C) interface to render results. The lesson is that structured, machine-readable audit artifacts should be designed into the pipeline architecture from the start, not added at the end.

7. Skills acquired through this project.

This project required and developed a range of skills across machine learning engineering, software architecture, and AI governance. On the technical side, proficiency was developed in LoRA fine-tuning using the Transformers and PEFT libraries, SLM deployment with Ollama, FastAPI microservice design, Pydantic schema validation, and Python-based evaluation pipeline construction. On the governance side, working knowledge was developed in EU AI Act provisions, NIST AI RMF, OWASP LLM Top 10, and GDPR data minimization requirements — knowledge acquired through the literature review and applied directly in the design of the Judge 2 governance evaluation protocols. Project management skills — particularly dependency management, risk mitigation through work reordering, and integration contract design — were developed through direct experience with the challenges described above.

13. Conclusions and Summary

This applied project successfully delivered Part (B) of the UNICC AI Safety Lab Capstone by implementing a fully operational Council of Experts ensemble for pre-deployment AI safety evaluation. The core contribution is a three-judge multi-module inference system — covering technical safety, governance and policy alignment, and operational risk — coordinated by an explicit critique and arbitration layer and synthesized into an actionable final verdict. The VeriMedia case study validated the system's practical value: three judges producing divergent assessments (Medium, Critical, and High) were reconciled into a unified Critical verdict that neither any individual judge nor a simple average would have produced, demonstrating the epistemic necessity of multi-perspective evaluation.

The project also produced a significant technical finding: while the base llama3.2:3b model achieved strong performance on the evaluation task (risk-tier match rates of 90% and 100% for Judges 1 and 2), the LoRA fine-tuned models produced schema-invalid outputs due to a training data format mismatch. This finding is documented transparently because it constitutes a substantive contribution: it identifies a specific, replicable failure mode in SLM fine-tuning pipelines — schema drift between training data and evaluation Pydantic models — that is not well-documented in the existing literature and that has a clear remediation path.

The project achieved all five defined objectives: integration protocols and API schemas were established on time; the core Compliance Judge was encapsulated as a FastAPI microservice; all three expert judges and the orchestration layer were implemented and validated; LoRA fine-tuning was completed and evaluated; and the deployment package was finalized with Dockerfile, Makefile, mock mode, and a validated test suite. The final status report received a green overall assessment and sponsor sign-off from Dr. Andres Fortino on April 17, 2026.

14. Limitations, Recommendations and Scope for Future Work

Limitations

Scope limited to Part (B). This project implemented the evaluation intelligence layer but did not build the platform infrastructure (Part A) or the user interface (Part C). The quality of the end-to-end system therefore depends on the quality of the platform delivered by Part (A) and the interface delivered by Part (C).

Fine-tuning schema mismatch. The LoRA fine-tuned models produced schema-invalid outputs due to a training data format inconsistency. As a result, the deployed system uses the base llama3.2:3b model rather than the fine-tuned versions. The evaluation protocols were therefore validated on the base model's capabilities, which — while strong — were not domain-adapted specifically for GRC evaluation tasks.

Evaluation dataset size. The held-out evaluation sets for Judge 1 and Judge 2 contain 10 examples each. This is sufficient to demonstrate the benchmark methodology and identify the schema-alignment issue, but it is not sufficient to draw statistically reliable conclusions about fine-tuned model performance. Larger evaluation sets would be required for production deployment decisions.

Judge 3 not fine-tuned. The LoRA fine-tuning pipeline was applied to Judges 1 and 2 only. Judge 3 (AdversarialProbe) was not included in the fine-tuning scope because its evaluation methodology — generating adversarial test cases rather than scoring structured inputs — requires a different training approach. Future work should extend the fine-tuning pipeline to cover Judge 3.

Recommendations and Future Work

Schema-lock the training data pipeline. Regenerate training datasets using a pipeline that imports the evaluation Pydantic models directly, ensuring that generated instruction-response pairs are always schema-compatible. Add an automated schema contract test to the pre-training workflow that validates a sample of generated outputs before compute is spent on fine-tuning.

Expand the evaluation dataset. Increase the held-out evaluation sets from 10 to at least 100 examples per judge, covering a broader range of risk tiers and deployment contexts. This is a prerequisite for production deployment and for meaningful comparison between fine-tuned and baseline performance.

Extend fine-tuning to Judge 3. Design a training approach for the AdversarialProbe module that generates adversarial test cases and evaluates pass/fail outcomes, rather than producing structured scoring assessments. This would require a different dataset generation methodology — possibly using existing red-team datasets or augmenting with synthetically generated adversarial examples.

Add longitudinal run history and comparison. A high-value enhancement is the ability to compare evaluation results across multiple runs of the same agent — before and after remediation — and to visualize trends in risk tier and score over time. This would transform the system from a single-run evaluation tool into a continuous governance monitoring capability.

Pursue AI Digital Public Good certification. Because the system uses open-weight models, open-source weights, and openly licensed code, it meets the eligibility criteria for AI Digital Public Good (AI DPG) classification under DPGA standards. Pursuing this certification would increase the system's visibility and potential for adoption by other international organizations.

15. Literature Survey

Student: Dongdi Sun

Organization: United Nations International Computing Centre (UNICC)

Date: April 1, 2026

1. Introduction

This literature survey synthesizes practice-based research relevant to delivering Part (B) of the UNICC AI Safety Lab Capstone: fine-tuning a Small Language Model and building a multi-module inference ensemble for pre-deployment AI safety evaluation. The review focuses on three thematic areas: (1) the AI threat landscape and governance frameworks that motivate multi-perspective evaluation; (2) ensemble architectures and observability practices that inform the Council of Experts design; and (3) SLM fine-tuning and LoRA adaptation techniques that underpin the domain adaptation methodology. Together, these themes support the core goal of Part (B): delivering an evaluation system that is technically sound, governance-aligned, and auditable.

2. The AI Threat Landscape

Cisco's State of AI Security 2026 report documents a transition from an 'AI-assisted era' to an 'AI-driven era' characterized by autonomous agents operating in continuous decision-action loops. In this paradigm, a single compromised instruction or misconfigured agent can cascade through multi-agent pipelines with minimal human visibility. The report identifies four threat categories particularly relevant to the UNICC context: prompt injection and jailbreaks, AI supply chain vulnerabilities, agentic threats arising from excessive agent authority, and data privacy risks including PII leakage and inference attacks.

The most salient empirical finding for this project is the multi-turn jailbreak vulnerability data. Across eight popular open-weight models — including Meta LLaMA 3.3-70B, DeepSeek v3.1, and Microsoft Phi-4 — single-turn jailbreak success rates ranged from 6.3 to 22.0 percent, while multi-turn success rates escalated dramatically, reaching up to 92.78 percent. This disparity indicates that static, single-pass safety checks are structurally inadequate; robust evaluation must stress-test models across extended interaction sequences. This finding directly motivated the adversarial red-team protocol architecture in Judge 3.

3. Governance and Regulatory Frameworks

No single regulatory framework provides comprehensive coverage of AI safety risks, which itself constitutes a major theoretical motivation for the multi-perspective ensemble approach. The EU AI Act, whose General Purpose AI provisions entered into force in August 2025, imposes mandatory adversarial testing, model evaluation, and incident reporting requirements on foundation models that could pose systemic risk. The NIST AI Risk Management Framework (AI RMF), updated in March 2025 to encompass adversarial machine learning threats, provides a voluntary but widely adopted blueprint for identifying, assessing, and monitoring AI-related risks. Both frameworks directly inform the scoring dimensions implemented in Judge 2, which

applies a weighted assessment model across five governance pillars.

OWASP's LLM Top 10 (2025 edition) provides a practical taxonomy of the most critical vulnerabilities in LLM-based applications, including prompt injection (ranked first), insecure output handling, and excessive agency. MITRE ATLAS provides an adversarial threat taxonomy specifically for AI systems. Together, these frameworks informed the protocol suites implemented across all three judges — no single framework covers all dimensions, which is itself the primary justification for the three-judge architecture.

4. Multi-Module Inference Ensembles

The Council of Experts architecture draws on a well-established principle in machine learning: ensembles of diverse, independently trained models consistently outperform single-model solutions on complex classification tasks. In the context of AI safety evaluation, diversity of perspective is not merely a performance optimization but an epistemic necessity. A system that scores well on technical robustness may fail on governance alignment, and vice versa.

Fiddler's guide to agentic observability articulates the underlying challenge precisely: multi-agent systems multiply complexity not linearly but exponentially, and 'without granular visibility, diagnosing issues becomes guesswork.' The guide argues that ensuring reliability across an agentic hierarchy requires monitoring every handoff, every intermediate decision, and every integration point. The Council architecture implements this principle by making each module's intermediate reasoning transparent and traceable: every judge produces structured output including a risk score, a risk tier, a confidence value, key findings, and protocol-level evidence, all of which are preserved in the pipeline log.

The critique round operationalizes a key insight from ensemble theory: explicit disagreement between evaluators contains information. Rather than averaging scores silently, the orchestrator surfaces agreement points, disagreement points, and arbitration notes before producing a reconciled score — consistent with Cisco's recommendation that AI safety systems 'surface disagreements between different safety assessment approaches, encouraging more thoughtful human oversight.'

5. SLM Fine-Tuning and LoRA Adaptation

The decision to build the AI Safety Lab on a standalone SLM rather than external API-based LLMs reflects both governance requirements and practical constraints. UNICC's AI Sandbox mandate requires that sensitive data not be used to train vendor models; an on-premises deployment provides the necessary data isolation and auditability. Domain-specific fine-tuning can substantially improve an SLM's performance on specialized evaluation tasks, compensating for the reduced parameter count relative to frontier models.

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique that updates only a small fraction of model weights by learning low-rank decompositions of weight update matrices. This makes fine-tuning feasible on constrained compute budgets — a critical requirement for the DGX Spark cluster. The technical literature on LoRA demonstrates consistent improvements over base model performance on domain-specific tasks when training data is well-aligned with the target evaluation format.

The benchmark results in this project — where fine-tuned models produced schema-invalid

outputs despite the base model achieving strong performance — underscore a practical lesson not always emphasized in the LoRA literature: schema alignment between training data and evaluation formats is a prerequisite for fine-tuning to be effective. This finding contributes a specific, replicable failure mode to the body of practice knowledge around SLM fine-tuning in structured-output applications.

6. AI Safety Evaluation Best Practices

HELM (Holistic Evaluation of Language Models) exemplifies a standardized approach that combines scenario coverage with multi-metric reporting to improve transparency of LLM behavior. For this project, HELM reinforces two practical design principles: evaluation should be organized around scenarios, and reporting formats must be consistent so results are comparable across runs. Anthropic's red-teaming research describes systematic probing to identify and reduce harmful outputs, with lessons learned from iterative mitigation cycles. From a design standpoint, red-teaming implies that evaluation tools should support organizing prompts and scenarios, tagging discovered failure modes, and making results reviewable by non-developer stakeholders.

Model Cards and Datasheets for Datasets propose standardized documentation formats that directly inform what the evaluation interface should surface: intended use, limitations, scenario coverage, evaluation settings, and applicable caveats. These documentation practices were applied in the design of the structured pipeline artifacts produced by the system.

7. Research Gap

The foregoing review reveals a clear gap: the absence of a pre-deployment, multi-perspective, domain-adapted evaluation system specifically designed for institutional governance contexts. Existing academic and industry work falls into three categories, each addressing only part of the problem: framework-level guidance provides taxonomies without deployable tools; commercial observability platforms focus on post-deployment monitoring rather than pre-deployment gatekeeping; and general-purpose red-teaming tools test specific vulnerability classes in isolation. The combination of four characteristics that the UNICC AI Safety Lab provides — pre-deployment scope, multi-perspective integration, domain adaptation via LoRA fine-tuning, and on-premises auditable infrastructure — is not available in any existing published solution.

8. Conclusion

The literature converges on a consistent message: successful AI safety evaluation is not only a modeling problem — it is also a governance, architecture, and documentation problem. The Council of Experts design, grounded in ensemble theory, adversarial threat research, and governance framework analysis, directly addresses the gap identified in the literature. The fine-tuning benchmark finding adds a specific practical contribution: schema alignment between training data and evaluation models is a first-order engineering requirement that must be enforced programmatically rather than assumed.

16. Situational Analysis

Student: Dongdi Sun

Organization: United Nations International Computing Centre (UNICC)

Date: March 26, 2026

1. Executive Summary

This situational analysis summarizes the external and internal environment influencing delivery of UNICC AI Safety Lab Capstone Part (B), which focuses on SLM fine-tuning and the Council of Experts multi-module inference ensemble. The analysis reviews industry context, the competitive landscape, and organizational culture. Porter's Five Forces and a SWOT analysis are applied to identify forces that may affect project success and to translate those forces into actionable delivery strategies.

2. Industry Analysis

UNICC operates as a technology and digital services provider for UN entities in a context where expectations for AI transparency, risk management, and evidence of evaluation are increasing rapidly. The EU AI Act, NIST AI RMF, and OWASP LLM Top 10 all impose or recommend systematic pre-deployment evaluation of AI systems — creating regulatory pull for exactly the kind of capability that the AI Safety Lab provides. At the same time, the pace of AI agent adoption is outrunning organizational readiness: Cisco's 2026 report found that 83% of organizations planned to deploy agentic AI capabilities but only 29% felt genuinely prepared to do so securely.

3. Competitive Landscape

In the capstone context, relevant substitute categories include open-source safety evaluation toolkits and benchmarks, commercial model monitoring and governance platforms, internal evaluation scripts and ad-hoc red-teaming processes, and other academic prototypes. To outperform these substitutes, Part (B) must emphasize multi-perspective coverage, structured and auditable output, and on-premises deployability — the specific combination that no existing solution provides.

4. Porter's Five Forces Analysis

- Competitive rivalry: High — many evaluation approaches exist; the Council of Experts differentiates through multi-perspective synthesis and on-premises auditability.
- Threat of new entrants: Medium — new evaluation tools appear quickly; modular architecture and documented APIs reduce the cost of future migration.
- Threat of substitutes: High — commercial governance platforms or single-judge LLM approaches could substitute; Part (B) adds value through the critique and arbitration layer.

- Bargaining power of buyers (stakeholders): High — UNICC can reject evaluation approaches that do not meet auditability or governance requirements; structured outputs and audit logs directly address this.
- Bargaining power of suppliers: Medium — dependencies include the DGX Spark cluster, the base SLM, and upstream Parts (A/B); LoRA and open-weight models reduce vendor lock-in.

5. SWOT Analysis

Strengths: clear three-judge architecture with distinct evaluation perspectives; LoRA fine-tuning pipeline for domain adaptation; Pydantic-validated structured outputs that are audit-ready; on-premises deployment eliminating external API dependencies.

Weaknesses: strong dependency on Part (A) for base SLM delivery; fine-tuning schema mismatch requiring dataset regeneration; evaluation dataset size insufficient for production deployment decisions.

Opportunities: differentiate through auditable, multi-perspective evaluation not available in existing tools; establish reusable GRC evaluation pipeline for future UN AI governance work; pursue AI Digital Public Good classification.

Threats: schema evolution in Parts (A) or (C) breaking integration; compute constraints limiting fine-tuning scope; regulatory framework evolution requiring protocol updates.

6. Implications for Project Strategy

Based on the forces above, Part (B) should prioritize: schema-first development — defining Pydantic models before writing any judge logic; audit artifact design — producing structured JSON at every pipeline stage; integration contract enforcement — programmatic schema contract tests rather than informal agreements; and baseline validation — confirming that the base model meets performance thresholds before investing compute in fine-tuning.

Appendices

Appendix A. Project Status Reports

Figure A-1. Midterm Status Report (Yellow — March 17, 2026)

PROJECT STATUS REPORT

PROJECT: UNICC AI Safety Lab — Project 2: Fine-Tuning the SLM and Building the Council of Experts

Overall Project Status	☐ No major issues	☒ Some major issues	☐ Significant issues
Explain The project is progressing smoothly. We have successfully encapsulated our core "Compliance Judge" expert module (built on Inspect AI) into an independent, architecture-compliant FastAPI microservice. The API is currently running stably in the local environment and successfully outputs structured JSON evaluation reports that align with the required governance dimensions. However, we are a bit behind schedule due to a delay with project 1.			

05 ProjectStatusReport v4.1.docx

Page 1 of 4

1-Schedule	☐ No major issues	☒ Some major issues	☐ Significant issues
Explain There was a slight delay in the integration timeline due to an unexpected personal emergency experienced by a member of the Project 1 team. However, the situation has been resolved, they are now back on track, and our overall project schedule has not been significantly impacted.			

2- Milestones	☒ No major issues	☐ Some major issues	☐ Significant issues
Explain Successfully achieved the critical milestone of "Expert Module Servitization." We have fully connected the end-to-end pipeline: taking standardized text input, running Language Model-based evaluations across international AI governance frameworks, and returning structured data outputs.			

3 - Project Deliverables	☒ No major issues	☐ Some major issues	☐ Significant issues
COMPLETED DELIVERABLES: Core expert evaluation logic specifically tailored for United Nations regulations, incorporating weighted scoring for UNESCO AI Ethics Recommendations, Humanitarian Compliance standards, and other international AI governance frameworks. FastAPI wrapper converting the local script into a real-time RESTful service. Internal module path routing and async execution adapters for the Inspect AI framework. UPCOMING DELIVERABLES: Creating a Dockerfile to containerize the API service for deployment to the DGX Spark or sandbox environment Developing the Coordination Layer (critique and arbitration logic) to synthesize outputs from multiple judges.			

4 – Issues	☐ No major issues	☒ Some major issues	☐ Significant issues
Explain There has been a delay in receiving the base SLM (Small Language Model) environment from the Project 1 team. Consequently, we have not yet been able to commence the fine-tuning phase of our project. To maintain project momentum and avoid idle time, we have pivoted our immediate focus to developing, refining, and encapsulating the expert agent module based on the previous group's foundation. We will resume the SLM fine-tuning as soon as Project 1 delivers the required model.			

6– Resources & Collaboration	☒ No major issues	☐ Some major issues	☐ Significant issues
Explain Despite the brief pause caused by the personal situation in the Project 1 team mentioned above, communication has resumed. Moving forward, we will work closely with them to verify their Request Manager's payload format and ensure our API endpoints are successfully registered.			

– Sponsor Signoff	
Sponsor indicates agreement with the above status report.	
<i>Andres Fortino</i>	
Signature	Name

Figure A-2. Final Status Report (Green — April 17, 2026)

PROJECT STATUS REPORT

UNICC AI Safety Lab — Project 2: Fine-Tuning the SLM and Building the Council of Experts

STATUS DETAILS

Report only on the status of your project, not on course assignment deliverables.

Overall Project Status	No major issues	Some major issues	Significant issues
Overall Project Status	X		
Explain	The project has been completed successfully. All planned components of the UNICC AI Safety Lab have been fully implemented, integrated, and tested. The complete end-to-end pipeline — from Streamlit frontend through FastAPI backend, three independent expert judges, a critique arbitration round, and a final synthesis layer — is fully operational. The system delivers transparent, auditable, multi-perspective AI safety evaluations and meets all project objectives.		

1 — Schedule	No major issues	Some major issues	Significant issues
1 — Schedule	X		
Explain	The temporary schedule delay noted at the midterm — caused by a personal emergency within the Project 1 team — was fully recovered. All remaining deliverables were completed on schedule. The final integration and testing phases were completed within the planned timeframe, with no net slippage to the overall project deadline.		

2 — Milestones	No major issues	Some major issues	Significant issues
Milestones	X		
Explain	All project milestones achieved: Expert Module Servitization: Compliance Judge encapsulated as an independent, architecture-compliant FastAPI microservice.		

	- Three Expert Judges Implemented: Judge 1 (Technical Safety), Judge 2 (Governance & Policy Alignment), Judge 3 (Operational & System Risk) — all fully operational and returning structured outputs. - Critique & Arbitration Layer: Reconciliation logic synthesizing multi-judge outputs, producing agreement/disagreement analysis and reconciled risk scores. - Full-Stack Integration: Streamlit frontend dynamically connected to FastAPI backend; user inputs flow through POST /submit and real-time results are rendered on the dashboard. - Local SLM Integration: Ollama (llama3.2:3b) configured and validated as the local model provider, fully replacing the dependency on external API keys for core judges. - Containerization & Deployment Readiness: Dockerfile and Makefile complete and tested for deployment to the DGX Spark or sandbox environment.
--	--

3 — Project Deliverables	No major issues	Some major issues	Significant issues
Deliverables	X		
Completed Deliverables	- Core expert evaluation logic for UN/UNESCO frameworks, with weighted scoring across international governance dimensions (UNESCO AI Ethics, Humanitarian Compliance, and related standards). - FastAPI microservice wrapping all three judges with real-time RESTful endpoints (POST /submit) and structured JSON responses. - Orchestrator module (orchestrator.py) coordinating the full multi-judge evaluation pipeline and critique round. - Synthesis layer (synthesis.py) producing final safety recommendation, risk tier, rationale, and recommended next actions. - Streamlit frontend (demo.py) fully connected to the FastAPI backend via HTTP, with dynamic result rendering of all judge and synthesis outputs. - Structured JSON output files: judge1_output.json, judge2_output.json, judge3_output.json, synthesis_output.json, full_result.json, and pipeline_log.json. - Dockerfile and Makefile for containerized deployment and one-command startup (make run) targeting the DGX Spark or sandbox environment. - Mock mode (MOCK_MODE=1) for CI/testing without live LLM dependencies, returning realistic pre-computed outputs. - Test suite (tests/test_scoring.py) and environment configuration template (.env.example).		

4 — Issues	No major issues	Some major issues	Significant issues
4 — Issues	X		
Explain	All issues identified at the midterm have been fully resolved. The SLM fine-tuning phase was successfully completed once the base model was delivered		

	by the Project 1 team. The system has been validated end-to-end with no outstanding blockers. All components are stable, and the pipeline handles graceful failure cases — such as missing API keys or Ollama not running — with clear error messaging.
--	---

6 — Resources & Collaboration	No major issues	Some major issues	Significant issues
6 — Resources & Collaboration	X		
Explain	Collaboration with the Project 1 team was successfully re-established. The Request Manager payload format was verified and our FastAPI endpoints were registered and confirmed compatible. All team members contributed to completing their respective modules on time. No resource or collaboration issues remain outstanding.		

Sponsor Signoff

Sponsor indicates agreement with the above status report.

Signature	Name
<i>Andres Fortino</i>	Prof Andres Frotino

Appendix B. Project Plan, Baseline WBS

Figure B-1. Work Breakdown Structure

WBS	Work Package	Key Activities
1.0	Architecture & Interface Planning	Define API schemas, risk scoring metrics, traffic-light thresholds, and cross-project interaction protocols with Projects 1 and 3.
2.0	Expert Layer Development	Implement Compliance Judge module; integrate UN/UNESCO AI Ethics, EU AI Act, and NIST AI RMF frameworks; develop prompt engineering and weighted scoring algorithms.
3.0	Microservice Encapsulation	Develop FastAPI application; create async evaluation adapter; implement OpenAPI (Swagger) endpoints; validate real-time RESTful output.
4.0	Coordination Layer Development	Design and implement critique/arbitration logic (orchestrator.py) and synthesis engine (synthesis.py) to reconcile multi-judge outputs into a final verdict.

5.0	SLM Fine-Tuning & Integration	Receive base SLM from Project 1; generate GRC-specific training datasets; execute LoRA fine-tuning; conduct end-to-end integration testing.
6.0	Deployment & Containerization	Write Dockerfile and Makefile; test containerized API locally; deploy to DGX Spark / sandbox environment with mock mode for CI.
7.0	Project Management & Documentation	Conduct bi-weekly check-ins; draft status reports; write API integration guides; prepare final presentation and handoff materials.

Deviation summary: The project experienced a mid-term delay in the SLM fine-tuning phase due to a personal emergency on the Project 1 team, which delayed delivery of the base SLM environment. This was recovered by reordering work packages to accelerate orchestration and synthesis development. Practical completion was achieved by April 14, 2026, with the final status report submitted on April 17, 2026. No scope changes were required to achieve the Part (B) goal and objectives.

Appendix C. References and Bibliography (APA)

Chang, A., et al. (2026). State of AI Security 2026. Cisco Systems. Retrieved from Cisco AI Security Research.

EU AI Office. (2025). EU AI Act: General Purpose AI Model Provisions (August 2025 Implementation). European Commission.

Fiddler AI. (2025). Rethinking Observability for the Age of AI Agents. Fiddler. Retrieved from fiddler.ai.

Fortino, A. (2026, January 12). Memorandum: Participate in the UNICC AI Safety Lab Project (MASY GC-4100). New York University School of Professional Studies.

Ganguli, D., Askell, A., Schiefer, N., Liao, T., Chen, A., Goldie, A., & Bowman, S. R. (2022). Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv:2209.07858. <https://arxiv.org/abs/2209.07858>

Gebru, T., Morgenstern, J. H., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for Datasets. *Communications of the ACM*, 64(12), 86–92. <https://doi.org/10.1145/3458723>

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations (ICLR 2022)*. <https://arxiv.org/abs/2106.09685>

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., & Koreeda, Y. (2022). Holistic Evaluation of Language Models. arXiv:2211.09110. <https://arxiv.org/abs/2211.09110>

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., & Gebru, T. (2019). Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, 220–229. <https://doi.org/10.1145/3287560.3287596>

National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1). U.S. Department of Commerce. <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>

NIST. (2025). NIST AI 100-2e2025: Adversarial Machine Learning Taxonomy Update. National Institute of Standards and Technology.

OWASP. (2025). OWASP LLM Top 10 (2025 Edition). Open Web Application Security Project.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28, 2503–2511.
<https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>